

QUEUE

Data Structure — Array & Linked List Implementations

Insertion • Deletion • Searching • Real-World Code Examples

Deque



What is a Queue?

A linear data structure that follows *FIFO* — First In, First Out



Definition

Elements are added at the **REAR** and removed from the **FRONT**. The element inserted first is the one removed first — just like a line of people at a ticket counter.



Core Operations

Enqueue — insert an item at the rear

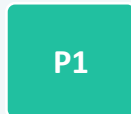
Dequeue — remove an item from the front

Peek / Front — view the front item

isEmpty / isFull — check queue status

Example: Ticket Counter Queue

FRONT
(served next)



REAR (joins here)

P1 arrived first → P1 is served first.

New person (P5) joins at the rear.

This is exactly how a queue behaves in memory — FIFO order, always.

Contrast with Stack: a Stack is LIFO (Last In, First Out) — like a stack of plates.

Types of Queue

Simple Queue
(this lab)

Circular Queue

Priority Queue

Deque
(Double-Ended)

Two Ways to Implement a Queue

Choosing the underlying storage: Array or Linked List

Array Implementation

- Fixed-size contiguous block of memory
- Uses two pointers: **front** and **rear**
- Fast, cache-friendly access
- Size must be decided in advance
- Simple array can “waste” freed slots

(solved by Circular Queue)



front=0

rear=2

Linked List Implementation

- Dynamic size — grows as needed
- Uses two pointers: **front** and **rear** (node pointers)
- No memory wasted, no fixed limit
- Needs extra memory for pointers
- No random-access indexing



front

rear

Array Implementation — Insertion (Enqueue)

Add a new element at the rear end of the array

Steps:

1. Check if the queue is full ($\text{rear} == \text{size} - 1$)
2. Increment rear by 1
3. Place new value at $\text{arr}[\text{rear}]$
4. If it is the very first element, set $\text{front} = 0$

Example: Enqueue(40) on [10, 20, 30]



front=0

rear=3 (new)

40 is inserted right after 30, at index 3 — the new rear.

```
#define SIZE 5
int arr[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        cout << "Queue is Full!";
        return;
    }
    if (front == -1) {
        front = 0;    // first element
    }
    rear++;
    arr[rear] = value;
    cout << value << " enqueued.";
}
```

Array Implementation — Deletion (Dequeue)

Remove the element from the front of the array

Steps:

1. Check if the queue is empty ($\text{front} == -1$)
2. Read the value at $\text{arr}[\text{front}]$
3. Increment front by 1
4. If $\text{front} > \text{rear}$, reset both to -1 (queue empty)

Example: Dequeue() on [10, 20, 30, 40]



front=0 new front=1 rear=3
(removed → 10)

10 leaves the queue first — the classic FIFO behavior.

```
int dequeue() {  
    if (front == -1 || front > rear) {  
        cout << "Queue is Empty!";  
        return -1;  
    }  
    int value = arr[front];  
    front++;  
  
    if (front > rear) {  
        front = rear = -1; // queue empty now  
    }  
    return value;  
}
```

Array Implementation — Searching an Element

Linear search between front and rear

To find a value, scan every slot from **front** to **rear** — a queue is not sorted, so binary search cannot be used. This is called

Linear Search, with time complexity $O(n)$.

```
int search(int key) {
    for (int i = front; i <= rear; i++) {
        if (arr[i] == key)
            return i;    // found: index
    }
    return -1;           // not found
}
```

Full Working Example (Array Queue)

```
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    cout << dequeue();    // removes 10

    int pos = search(30);
    if (pos != -1)
        cout << "Found at " << pos;
    else
        cout << "Not found";
    return 0;
}
```

Linked List Implementation — Insertion (Enqueue)

Attach a new node after the current rear node

Steps:

1. Create a new node, store the value, next = NULL
2. If queue is empty: front = rear = new node
3. Otherwise: rear->next = new node
4. Move rear to the new node

Node structure

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node *front = NULL, *rear = NULL;
```

New node (40) links after rear (30):



```
void enqueue(int value) {  
    Node* temp = new Node();  
    temp->data = value;  
    temp->next = NULL;  
  
    if (rear == NULL) {  
        // queue was empty  
        front = rear = temp;  
        return;  
    }  
    rear->next = temp;  
    rear = temp;  
    cout << value << " enqueued."  
}
```

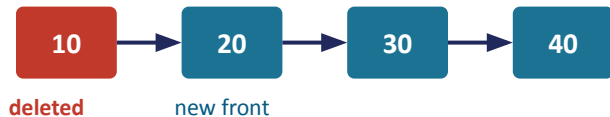
Linked List Implementation — Deletion (Dequeue)

Remove the front node and free its memory

Steps:

1. Check if front == NULL (queue empty)
2. Keep a temp pointer to the front node
3. Move front to front->next
4. Delete the temp node (free memory)
5. If front becomes NULL, set rear = NULL too

10 is removed from the front:



```
int dequeue() {
    if (front == NULL) {
        cout << "Queue is Empty!";
        return -1;
    }
    Node* temp = front;
    int value = temp->data;

    front = front->next;
    if (front == NULL)
        rear = NULL;    // list became empty

    delete temp;
    return value;
}
```

Linked List Implementation — Searching an Element

Traverse node by node from front until found

Starting at **front**, follow each node's **next** pointer and compare data with the key. Also $O(n)$ time, but no direct indexing is possible — only sequential traversal.

```
bool search(int key) {
    Node* current = front;
    int position = 0;
    while (current != NULL) {
        if (current->data == key) {
            cout << "Found at " <<
position;
            return true;
        }
        current = current->next;
        position++;
    }
    return false;    // not found
}
```

Full Working Example (Linked List Queue)

```
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    cout << dequeue();    // removes 10

    if (search(30))
        cout << "30 is in the queue";
    else
        cout << "Not found";
    return 0;
}
```

Why Queues Matter — Real-World Applications

FIFO ordering shows up everywhere in computing



CPU Scheduling

OS keeps ready processes in a queue; each gets CPU time in the order it arrived (Round-Robin scheduling).



Printer / Task Spooling

Print jobs are queued and printed in the exact order they were sent by users.



Handling Requests

Web servers queue incoming requests and serve them one by one, keeping order fair (e.g., call center systems).



Breadth-First Search (BFS)

Graph and tree traversal algorithms use a queue to visit nodes level by level.



Data Buffers

Keyboard input buffers, video streaming buffers, and IO buffers all rely on queue behavior.



Online Ticket Booking

Users are placed in a virtual queue during high traffic, served strictly in arrival order.

Summary — Array vs Linked List Queue

Aspect	Array Queue	Linked List Queue
Size	Fixed at creation	Dynamic / grows as needed
Memory	Contiguous, may waste slots	Non-contiguous, uses pointers
Insertion (Enqueue)	$O(1)$ — rear++	$O(1)$ — link at rear
Deletion (Dequeue)	$O(1)$ — front++	$O(1)$ — unlink at front
Search	$O(n)$, index accessible	$O(n)$, sequential only
Best for	Known, limited capacity	Unknown / changing capacity